

สร้างระบบ Dashboard และแจ้งเตือนอัตโนมัติ บน Pi5

เส้นทางข้อมูล: DHT22 → Python → Node-RED → InfluxDB (Docker) → Grafana → LINE / Email

สารบัญ

1. ต่อ DHT22 กับ Pi5
2. ติดตั้งและทดสอบสคริปต์ Python
3. ติดตั้ง InfluxDB ด้วย Docker
4. Node-RED: อ่าน DHT22 → เขียนลง InfluxDB
5. ติดตั้ง Grafana
6. เชื่อม Grafana กับ InfluxDB
7. สร้าง Dashboard แรก + Mobile Dashboard
8. Alert Rule / Contact Point / Notification Policy
9. Node-RED: Webhook → LINE

1. ต่อ DHT22 กับ Pi5

ขา DHT22	ต่อไปที่
VCC	Pi5 3.3V
DATA	Pi5 GPIO4 (= physical pin 7, BCM numbering)
GND	Pi5 GND

เช็ค pin mapping จริงด้วยคำสั่ง:

```
pinout
```

2. ติดตั้งและทดสอบสคริปต์ Python จาก Terminal

```
sudo apt-get install -y python3-pip
pip3 install adafruit-circuitpython-dht --break-system-packages
```

สร้างสคริปต์อ่านค่าเซนเซอร์:

```
cat > ~/dht_read.py << 'EOF'
import board
import adafruit_dht
import json
dht = adafruit_dht.DHT22(board.D4, use_pulseio=False)
try:
    print(json.dumps({"temperature": dht.temperature, "humidity": dht.humidity}))
except RuntimeError as e:
    print(json.dumps({"error": str(e)}))
finally:
    dht.exit()
EOF
```

ทดสอบรัน:

```
python3 ~/dht_read.py
```

ผลลัพธ์ที่ควรเห็น: {"temperature": 29.5, "humidity": 83.2} RuntimeError เป็นบางครั้งถือว่าปกติ ลองรันซ้ำได้ (เว้นอย่างน้อย 2 วิ/ครั้ง)

3. ติดตั้ง InfluxDB ด้วย Docker บน Pi5

ติดตั้ง Docker

```
curl -sSL https://get.docker.com | sh
sudo usermod -aG docker $USER
```

รันเสร็จให้ reboot เครื่อง แล้วเช็คด้วย `docker --version`

รัน InfluxDB container

```
docker run -d \
  --name influxdb \
  -p 8086:8086 \
  -v influxdb-data:/var/lib/influxdb2 \
  -v influxdb-config:/etc/influxdb2 \
  --restart unless-stopped \
  influxdb:2
```

ถ้าเจอ error "Conflict ... name already in use"

```
docker ps -a          # เช็ค container เดิมมีอยู่ไหม สถานะอะไร
docker start influxdb # ถ้า Exited อยู่ สั่งให้รันใหม่

# หรือถ้าอยากเริ่มใหม่จริงๆ (ข้อมูลใน volume ไม่หายเพราะแยกจาก container)
docker rm -f influxdb
# แล้วรัน docker run ด้านบนซ้ำอีกที
```

ตั้งค่าเริ่มต้นผ่านเบราว์เซอร์

เปิด `http://<pi5-ip>:8086` → Setup Initial User:

- Username / Password
- Organization Name
- Bucket Name
- ระบบสร้าง API Token ให้อัตโนมัติ → เก็บไว้ใช้ต่อ

ค่าจริงที่ใช้อยู่ตอนนี้:

ค่า	ตั้งไว้เป็น
Organization	xxxxxxxxxx
Bucket	xxxxxxxxxx

4. Node-RED Flow — เรียก Python อ่าน DHT22 แล้วเขียนลง InfluxDB

Flow: `exec (python3 dht_read.py)` → `json` → `function` → `influxdb out` ×2

exec node

ช่อง	ค่า
Command	<code>python3 /home/<user>/dht_read.py</code> (เช่น <code>/home/finh/dht_read.py</code>)
Output	when the command completes

json node

ไม่ต้องตั้งค่าอะไร แปลง JSON string เป็น object อัตโนมัติ

function node

```
const t = msg.payload.temperature;
const h = msg.payload.humidity;
return [
  { payload: { value: t } },
  { payload: { value: h } }
];
```

influxdb out node (บน) — สำหรับ temperature

ช่อง	ค่า
Name	temperature
Server	[v2.0] http://localhost:8086
Organization	xxxxxxxxxx
Bucket	xxxxxxxxxx
Measurement	temperature ← ต้องพิมพ์เองในช่องนี้ ไม่ได้มาจาก payload

influxdb out node (ล่าง) — สำหรับ humidity

เหมือนกันทุกอย่าง แต่ Measurement: **humidity**

เช็คข้อมูลถูกต้อง: InfluxDB UI → Data Explorer → bucket xxxxxxxxxx ควรเห็น `_measurement = temperature/humidity` สะอาดๆ และ `_field = value` ตัวเดียว (ถ้าเห็น `_field` เป็นตัวเลข 0, 1, 2... แปลว่า payload format ผิด กลับไปเช็ค function node ด้านบนว่ายังเป็น array อยู่หรือเปล่า)

5. ติดตั้ง Grafana บน Pi5

```
sudo apt-get install -y apt-transport-https wget
wget -q -O - https://apt.grafana.com/gpg.key | \
  gpg --dearmor | sudo tee /etc/apt/keyrings/grafana.gpg
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] \
  https://apt.grafana.com stable main" | sudo tee \
  /etc/apt/sources.list.d/grafana.list

sudo apt-get update && sudo apt-get install -y grafana
sudo systemctl enable --now grafana-server
```

เข้าใช้งานที่ <http://<pi5-ip>:3000> (เริ่มต้น **admin** / **admin**)

ถ้าเข้าจากเครื่องอื่นในวง LAN เดียวกันไม่ได้

```
sudo systemctl status grafana-server # ต้อง active (running)
hostname -I # เช็ค IP จ้างตรงกับที่ใช้เปิดไหม
sudo ss -tlnp | grep 3000 # ต้องเห็น *:3000 ไม่ใช่ 127.0.0.1:3000
sudo ufw status # ถ้า active ต้องมี 3000/tcp ในลิสต์ ALLOW
sudo ufw allow 3000/tcp # ถ้ายังไม่ให้เปิดพอร์ต

# เพื่อพอร์ตอื่นที่ใช้ในระบบนี้ด้วย
sudo ufw allow 1880/tcp # Node-RED
sudo ufw allow 8086/tcp # InfluxDB UI
```

6. เชื่อม Grafana เข้ากับ InfluxDB

Grafana → Connections → Data sources → Add data source → InfluxDB

ช่อง	ค่า
Query language	Flux
URL	<code>http://localhost:8086</code>
Organization	<code>xxxxxxxxxx</code>
Token	จาก Step 3
Default Bucket	<code>xxxxxxxxxx</code>

กด **Save & test** ต้องขึ้น "datasource is working"

ถ้าขึ้น "unauthorized: unauthorized access error reading buckets"

- เช็คค่า Query language = Flux (ไม่ใช่ InfluxQL)
- ปิด toggle **Basic auth** ถ้าเปิดอยู่ (ใช้ Org/Token ในช่อง InfluxDB Details ด้านล่างแทน)
- คัดลอก token ใหม่จาก terminal/หน้าเว็บ ระวัง whitespace/newline แอบติด
- ทดสอบ token แยกจาก Grafana ก่อนด้วย curl:

```
curl -X POST http://localhost:8086/api/v2/query \
-H "Authorization: Token <TOKEN>" \
-H "Content-Type: application/vnd.flux" \
-d 'buckets()'
```

7. สร้าง Dashboard แรก

Dashboards → New → New Dashboard → Add visualization → เลือก data source InfluxDB ที่เพิ่งเชื่อมต่อ

```
from(bucket: "iot_dataV2")
|> range(start: -1h)
|> filter(fn: (r) => r._measurement == "temperature" and r._field == "value")
```

ทำ panel ที่สองซ้ำแบบเดียวกัน เปลี่ยนแค่ `_measurement == "humidity"`

ถ้า panel วางไม่มีข้อมูล ไปเช็คตามลำดับ

1. เช็คก่อนว่ามีข้อมูลจริงใน InfluxDB Data Explorer ไหม (bucket ให้ตรง)
2. ขยายช่วงเวลา (time range) มุมขวาบนของ dashboard เป็น Last 24h
3. เช็คชื่อ bucket/measurement ใน query ให้ตรงกับที่ Node-RED เขียนจริง (พิมพ์เอง อย่า copy ตัวอักษร เพื่อ smart quote จากที่อื่นติดมา)
4. สลับไปดู query โหมด Code เทียบกับ panel ที่ใช้งานได้แล้ว — ถ้าจะทำ panel ใหม่ ให้ copy query จาก panel ที่ใช้ได้ มาแก้ไขชื่อ measurement แทนพิมพ์ใหม่ทั้งหมด

Mobile Dashboard (เข้าจากนอกบ้านผ่าน Tailscale)

```
sudo curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
```

รัน `tailscale up` แล้วจะขึ้น URL ให้เปิดใน browser เพื่อ login ด้วยบัญชี Google

จากนั้น: 1. โหลดแอป Tailscale จาก App Store (iOS) หรือ Play Store (Android) 2. เปิดแอปแล้ว login ด้วยบัญชีเดียวกันกับที่ใช้ตอนติดตั้งบน Pi5 3. เช็คความเห็นชื่อ Pi5 ขึ้นอยู่ในลิสต์เครื่องที่เชื่อมต่อ 4. เปิด browser พิมพ์ `http://100.x.x.x:3000` ตาม Tailscale IP ที่จดไว้

8.1 สร้าง Alert Rule แบบละเอียด

Alerting → Alert rules → New alert rule

1. Name: เช่น `temp_high`
2. Define query: เลือก data source InfluxDB แล้วเขียน Flux query จริงในช่องนี้ (ไม่ใช่ syntax WHEN/IS ABOVE — อันนั้นไปกรอกที่ Alert condition กล่องถัดไปต่างหาก)

```
from(bucket: "iot_dataV2")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "temperature" and r._field == "value")
```

3. Alert condition (กล่องด้านล่าง คนละกล่องกับ query):

```
WHEN Last OF QUERY IS ABOVE 40
```

4. Set evaluation behavior:

- ต้องเลือก/สร้าง Folder ก่อน (เช่น **IoT Alerts**)
- ต้องเลือก/สร้าง Evaluation group ก่อน (เช่น **sensor-checks**, interval **1m**)
- Pending period: **2m**

5. Labels / Annotations: ข้ามได้ ไม่บังคับ

6. Configure notifications: ปลอ่ย Contact point วางไว้ได้ (จะไปใช้ Default Notification Policy แทน ตามข้อ 8.3)

7. Configure notification message: ทั้งหมด optional ข้ามได้เลย

กด Preview alert rule condition เช็คก่อน แล้ว Save rule and exit

8.2 สร้าง Contact Point แบบ Webhook

ในบาง Grafana เวอร์ชัน เมนูอยู่ที่ Alerting → Notification configuration → แท็บ Contact points → Create contact point

ช่อง	ค่า
Name	node-red-webhook
Integration	Webhook (เปลี่ยนจาก Email ที่เป็นค่า default)
URL	http://localhost:1880/grafana-alert
HTTP Method	POST

หมายเหตุ: localhost ในช่องนี้ = เครื่องที่ Grafana รันอยู่ (คือ Pi5) ไม่ใช่เครื่อง PC ที่เปิดเบราว์เซอร์ดู ถ้า Node-RED อยู่เครื่องเดียวกับ Grafana ใช้ localhost ได้ถูกต้องแล้ว

กด Test ก่อน แล้วค่อยกด Save contact point

- Test ขึ้น 404 "Cannot POST /grafana-alert" = ปกติ ถ้ายังไม่ได้สร้าง flow ผัง Node-RED (ข้อ 9) — ทำต่อแล้วจะหาย
- Test ขึ้น timeout "context deadline exceeded" = flow ผัง Node-RED มีแล้วแต่ไม่มี http response node ตอบกลับ (ดูข้อ 9)

8.3 ผูก Notification Policy

Alerting → Notification configuration → แท็บ Notification policies → หา "Default policy" ด้านบนสุด → กด Edit → เปลี่ยน Default contact point จาก **grafana-default-email** เป็น **node-red-webhook** → Update default policy

Timing ของ Default policy (ค่า default ของ Grafana)

ตัวจับเวลา	ค่า default	ความหมาย
Group wait	30 วินาที	รอก่อนส่งแจ้งเตือนครั้งแรกของ alert ใหม่
Group interval	5 นาที	รวมส่งถ้ามี alert อื่นเข้า/ออกกลุ่มเดิม
Repeat interval	4 ชั่วโมง	ส่งซ้ำถ้า alert ยัง Firing ค้างไม่เปลี่ยน

ปรับ Repeat interval ให้สั้นลงได้ถ้าต้องการแจ้งถี่ขึ้นตอน Firing ค้าง Δ ระวัง: ตั้งสั้นเกินไปตอน threshold ต่ำกว่าจริงจะโดนสแปมรัว

เช็คไม่ให้มี Silence บังเอิญบล็อกอยู่: Alerting → Silences → ต้องไม่มีรายการค้าง

เช็คเว็บว่า webhook ส่งสำเร็จจริงหรือยัง: Alerting → Notification configuration → Contact points → ดูสถานะได้ **node-red-webhook** เช่น "Last delivery attempt failed" → กด Test ซ้ำเพื่อดูสถานะล่าสุด (สถานะเก่าอาจค้างจากรอบก่อน)

9. Node-RED Flow — Webhook สู่ LINE

9.1 เตรียม LINE Messaging API

1. สร้าง LINE Official Account ที่ manager.line.biz
2. เปิด Messaging API: OA Manager → Settings → Messaging API → Enable
3. LINE Developers Console (developers.line.biz) → Provider → Channel
4. ดัดลอก Channel Access Token (Messaging API tab → Issue) > ⚠ ห้ามแชร์ token นี้ที่ไหนก็ตามที่ไม่ปลอดภัย ถ้าหลุดไปแล้วให้ Issue ใหม่ทันทีเพื่อยกเลิกตัวเก่า
5. เพิ่ม OA เป็นเพื่อนด้วย QR code แล้วหา User ID / Group ID ปลายทาง

9.2 http in node

ช่อง	ค่า
Method	POST
URL	/grafana-alert

URL เติมต้องตรงกับที่กรอกใน Contact point (ข้อ 8.2)

http response node

ปล่อยค่า default ทั้งหมดไว้ได้เลย ไม่ต้องตั้งอะไร — ต่อสายตรงจาก output ของ **http in** มาที่ node นี้เท่านั้น

9.3 function node

```
const alerts = msg.payload.alerts || [];  
  
const text = alerts.map(a => {  
  const status = a.status === "firing" ? "ALERT" : "RESOLVED";  
  const name = a.labels.alertname;  
  
  let temp = "N/A";  
  if (a.values && a.values.B !== undefined) {  
    temp = a.values.B;  
  } else if (a.valueString) {  
    const match = a.valueString.match(/value=([d.]+)/);  
    if (match) temp = match[1];  
  }  
  
  return `${status}: ${name}\nTemperature: ${temp}°C`;  
}).join("\n\n");  
  
msg.payload = {  
  to: "<LINE_USER_OR_GROUP_ID>",  
  messages: [{ type: "text", text }]  
};  
msg.headers = {  
  "Content-Type": "application/json",  
  "Authorization": "Bearer <CHANNEL_ACCESS_TOKEN>"  
};  
return msg;
```

ตัวอย่างข้อความที่จะได้รับจริง:

```
ALERT: temp high  
Temperature: 31.1°C
```

9.4 http request node

ช่อง	ค่า	หมายเหตุ
Method	POST	ค่า default เป็น GET ต้องเปลี่ยน
URL	https://api.line.me/v2/bot/message/push	
Payload	ต้องไม่ใช่ "Ignore"	ค่า default คือ Ignore = ไม่ส่ง body เลย ต้องเปลี่ยนเป็นตัวเลือกที่ส่ง msg.payload
Return	a UTF-8 string	ค่า default โอเคอยู่แล้ว
Use authentication	ไม่ต้องตั้ง	auth ทำผ่าน msg.headers จาก function node แล้ว
Headers (ตาราง)	ปล่อยให้ว่าง ไม่ต้องกรอก	headers มาจาก msg.headers อัตโนมัติ ใส่ค่าเสี่ยงไปทับค่าที่ตั้งไว้

ทดสอบ

1. ต่อ debug node เข้ากับ output ของ http request ดู response จริง
2. Deploy
3. Grafana → Contact points → node-red-webhook → Test
4. เช็คมือถือว่าได้รับ LINE จริงไหม